

Dataframe Basic Operations (Python)

[Import notebook](#)

Creating a SparkSession

- The SparkSession is the entry point to programming with Spark SQL.
- It allows you to create DataFrames, register DataFrames as tables, execute SQL over tables, cache tables, and read parquet files.
- **SparkSession.builder**: The builder attribute is a class attribute of SparkSession that provides a way to configure and create a SparkSession instance.
- **appName("Example App")**: The appName method sets the name of the Spark application. This name will appear in the Spark web UI and can help you identify your application among others running on the same cluster.
- **config("spark.some.config.option", "some-value")**: The config method allows you to set various configuration options for the Spark session. In this example, "spark.some.config.option" is a placeholder for an actual configuration key, and "some-value" is the value for that configuration. You can set multiple configuration options by chaining multiple config calls.
- **getOrCreate()**: The getOrCreate method either retrieves an existing SparkSession if one already exists or creates a new one if it does not. This ensures that you do not accidentally create multiple SparkSession instances in your application.

Note:In Databricks, you do not need to create or override the SparkSession as it is automatically created for each notebook or job executed against the cluster. Databricks manages the SparkSession and SparkContext for you, ensuring optimal configuration and resource usage.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Spark DataFrames").config("spark.some.config.option", "some-value").getOrCreate()
```

Creating DataFrame

1.From Python a List of Tuples

```
%python
# List of tuples
data = [("John", 25), ("Doe", 30), ("Jane", 22)]

# Creating DataFrame
df_list = spark.createDataFrame(data, ["Name", "Age"])

# Display the DataFrame
df_list.show()
```

```
df_list: pyspark.sql.dataframe.DataFrame = [Name: string, Age: long]
```

```
+---+---+
|Name|Age|
+---+---+
|John| 25|
|Doe| 30|
|Jane| 22|
+---+---+
```

2.From a List of Dictionaries

```
%python
# List of dictionaries
data = [{"Name": "Alice", "Id": 1}, {"Name": "Bob", "Id": 2}, {"Name": "Cathy", "Id": 3}]

# Creating DataFrame
df_dict = spark.createDataFrame(data)

# Display the DataFrame
df_dict.show()
```

```
df_dict: pyspark.sql.dataframe.DataFrame = [Id: long, Name: string]
```

```
+---+---+
| Id| Name|
+---+---+
| 1|Alice|
| 2| Bob|
| 3|Cathy|
+---+---+
```

3.From a List of Rows

```
%python
from pyspark.sql import Row

# List of Rows
data = [ Row(Name="Cathy", Id=1),
         Row(Name="David", Id=2),
         Row(Name="Eva", Id=3),
         Row(Name="Frank", Id=4)]

# Creating DataFrame
df_row = spark.createDataFrame(data)

# Display the DataFrame
df_row.show()
```

```
df_row: pyspark.sql.dataframe.DataFrame = [Name: string, Id: long]
```

```
+---+---+
| Name| Id|
+---+---+
```

```
|Cathy| 1|
|David| 2|
|  Eva| 3|
|Frank| 4|
+-----+
```

4. Creating a DataFrame from an RDD

```
%python
# Import necessary modules
from pyspark.sql import Row

# Create an RDD
rdd = spark.sparkContext.parallelize([
    Row(Name="Alice", Age=25),
    Row(Name="Bob", Age=30),
    Row(Name="Cathy", Age=22),
    Row(Name="David", Age=35),
    Row(Name="Eva", Age=28),
    Row(Name="Frank", Age=40)
])

# Convert RDD to DataFrame
df_rdd = spark.createDataFrame(rdd)

# Display the DataFrame
df_rdd.show()
```

▶  df_rdd: pyspark.sql.dataframe.DataFrame = [Name: string, Age: long]

```
+-----+
| Name|Age|
+-----+
|Alice| 25|
|  Bob| 30|
|Cathy| 22|
|David| 35|
|  Eva| 28|
|Frank| 40|
+-----+
```

5. Reading external file

spark.read: This is the entry point for reading data in Spark. It returns a DataFrameReader object that is used to read data from various sources.

.format("csv"): Specifies the format of the data source. In this case, it indicates that the data is in CSV (Comma-Separated Values) format.

.option("header", "true"): This option tells Spark that the first row of the CSV file contains the column names. If this option is set to false, Spark will treat the first row as data. "true" means that the CSV file has a header row.

.option("inferSchema", "true"): This option tells Spark to automatically infer the data types of each column in the CSV file. If this option is set to false, all columns will be read as strings (default behavior). "true" means that Spark will try to infer the schema (data types) of the columns based on the data.

.load("/FileStore/tables/retail_db/customers"):

This method specifies the path to the CSV file or directory containing CSV files that you want to read.

```
customer_df=spark.read.format("csv").option("header","true").option("inferSchema","true").load("dbfs:/FileStore/tables/customers_300mb.csv")
```

▶  customer_df: pyspark.sql.dataframe.DataFrame = [customer_id: integer, name: string ... 5 more fields]

6. Using StructType & StructField

```
%python
#employee data and schemas
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, FloatType, DateType
from datetime import date

# Create dummy data as a list of lists
emp_data = [
    [1, 101, "John Doe", 30, "M", 60000.0, date(2020, 1, 15)],
    [2, 102, "Jane Smith", 25, "F", 65000.0, date(2019, 3, 10)],
    [3, 101, "Mike Johnson", 35, "M", 70000.0, date(2018, 5, 20)],
    [4, 103, "Emily Davis", 28, "F", 72000.0, date(2021, 7, 30)],
    [5, 102, "Robert Brown", 40, "M", 80000.0, date(2017, 9, 25)],
    [6, 101, "Linda Wilson", 32, "F", 68000.0, date(2020, 11, 5)],
    [7, 103, "David Lee", 29, "M", 75000.0, date(2019, 12, 15)]]

# Define the schema
emp_schema = StructType([
    StructField("empid", StringType(), True),
    StructField("deptid", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True),
    StructField("gender", StringType(), True),
    StructField("salary", FloatType(), True),
    StructField("hiredate", DateType(), True)
])

# Create DataFrame
df = spark.createDataFrame(emp_data, emp_schema)
#df = spark1.createDataFrame(data = emp_data, schema = emp_schema)

# Display the DataFrame
df.show()
```

▶  df: pyspark.sql.dataframe.DataFrame = [empid: string, deptid: integer ... 5 more fields]

```
+-----+-----+-----+-----+-----+-----+
|empid|deptid|      name|age|gender| salary| hiredate|
+-----+-----+-----+-----+-----+-----+
|   1|   101|  John Doe| 30|    M|60000.0|2020-01-15|
|   2|   102| Jane Smith| 25|    F|65000.0|2019-03-10|
|   3|   101|Mike Johnson| 35|    M|70000.0|2018-05-20|
```

```
| 4| 103| Emily Davis| 28| F|72000.0|2021-07-30|
| 5| 102|Robert Brown| 40| M|80000.0|2017-09-25|
| 6| 101|Linda Wilson| 32| F|68000.0|2020-11-05|
| 7| 103| David Lee| 29| M|75000.0|2019-12-15|
+-----+-----+-----+-----+-----+-----+-----+
```

Basic DataFrame Operation

1. show() & display()

In Databricks, `show()` and `display()` are used to visualize DataFrames, but they have different functionalities:

show(): This is a method available on Spark DataFrames that prints the first n rows to the console. It is useful for quick inspection of data but does not provide rich formatting or interactivity. You can specify the number of rows to display, and it defaults to 20 rows if not specified.

display(): This is a Databricks-specific function that provides a rich, interactive view of the DataFrame. It is more suitable for use within notebooks as it allows for better visualization, including sorting, filtering, and graphical representation of data.

```
customer_df.show(5)
```

```
+-----+-----+-----+-----+-----+-----+-----+
|customer_id|  name| city|      state|country|registration_date|is_active|
+-----+-----+-----+-----+-----+-----+-----+
|      0|Customer_0| Pune|Maharashtra| India|    2023-01-19|    true|
|      1|Customer_1| Pune|West Bengal| India|    2023-08-10|    true|
|      2|Customer_2| Delhi|Maharashtra| India|    2023-08-05|    true|
|      3|Customer_3| Mumbai| Telangana| India|    2023-06-04|    true|
|      4|Customer_4| Delhi| Karnataka| India|    2023-03-15|   false|
+-----+-----+-----+-----+-----+-----+-----+
```

only showing top 5 rows

```
customer_df.display()
```

```
#display(customer_df)
```

	customer_id	name	city	state	country	registration_date	is_active
1	0	Customer_0	Pune	Maharashtra	India	2023-01-19	true
2	1	Customer_1	Pune	West Bengal	India	2023-08-10	true
3	2	Customer_2	Delhi	Maharashtra	India	2023-08-05	true
4	3	Customer_3	Mumbai	Telangana	India	2023-06-04	true
5	4	Customer_4	Delhi	Karnataka	India	2023-03-15	false
6	5	Customer_5	Kolkata	West Bengal	India	2023-08-19	true
7	6	Customer_6	Kolkata	Tamil Nadu	India	2023-04-21	false
8	7	Customer_7	Mumbai	Telangana	India	2023-05-23	true
9	8	Customer_8	Pune	Tamil Nadu	India	2023-07-17	true
10	9	Customer_9	Delhi	Karnataka	India	2023-06-02	true
11	10	Customer_10	Hyderabad	Delhi	India	2023-02-23	true
12	11	Customer_11	Delhi	West Bengal	India	2023-11-08	true
13	12	Customer_12	Delhi	Delhi	India	2023-06-27	false
14	13	Customer_13	Pune	Maharashtra	India	2023-02-03	true
15	14	Customer_14	Chennai	Karnataka	India	2023-04-06	true

	customer_id	name	city	state	country	registration_date	is_active
14	13	Customer_13	Pune	Maharashtra	India	2023-02-03	true
15	14	Customer_14	Chennai	Karnataka	India	2023-04-06	true
16	15	Customer_15	Hyderabad	West Bengal	India	2023-03-31	true
17	16	Customer_16	Chennai	Maharashtra	India	2023-04-26	true
18	17	Customer_17	Pune	Delhi	India	2023-04-14	false
19	18	Customer_18	Chennai	Maharashtra	India	2023-02-04	false
20	19	Customer_19	Chennai	Karnataka	India	2023-01-22	true
21	20	Customer_20	Pune	Karnataka	India	2023-02-19	false
22	21	Customer_21	Hyderabad	Tamil Nadu	India	2023-09-16	true
23	22	Customer_22	Delhi	Karnataka	India	2023-12-27	true
24	23	Customer_23	Chennai	Tamil Nadu	India	2023-01-22	true
25	24	Customer_24	Pune	West Bengal	India	2023-09-11	true
26	25	Customer_25	Hyderabad	West Bengal	India	2023-08-22	true
27	26	Customer_26	Kolkata	West Bengal	India	2023-07-21	false
28	27	Customer_27	Pune	Gujarat	India	2023-02-09	false

10,000+ rows | Truncated data | 0.89s runtime

2. Columns & PrintSchema()

In Spark, `columns` and `printSchema()` are used to inspect the structure of a `DataFrame`, but they serve different purposes:

- **columns:** This attribute returns a list of the column names in the `DataFrame`.
- **printSchema():** This method prints the schema of the `DataFrame`, including column names and data types, in a tree format.

```
customer_df.columns
```

```
['customer_id',
 'name',
 'city',
 'state',
 'country',
 'registration_date',
 'is_active']
```

```
customer_df.printSchema()
```

```
root
|-- customer_id: integer (nullable = true)
|-- name: string (nullable = true)
|-- city: string (nullable = true)
|-- state: string (nullable = true)
|-- country: string (nullable = true)
|-- registration_date: date (nullable = true)
|-- is_active: boolean (nullable = true)
```

3. Select specific columns

```
customer_df.select("name", "city").show()
```

```

| Customer_5| Kolkata|
| Customer_6| Kolkata|
| Customer_7| Mumbai|
| Customer_8| Pune|
| Customer_9| Delhi|
|Customer_10|Hyderabad|
|Customer_11| Delhi|
|Customer_12| Delhi|
|Customer_13| Pune|
|Customer_14| Chennai|
|Customer_15|Hyderabad|
|Customer_16| Chennai|
|Customer_17| Pune|
|Customer_18| Chennai|
|Customer_19| Chennai|
+-----+-----+
only showing top 20 rows

```

4. Filter rows

```
customer_df.filter(customer_df.city=="Hyderabad").show()
```

```

|      21| Customer_21|Hyderabad| Tamil Nadu| India|      2023-09-16|      true|
|      25| Customer_25|Hyderabad|West Bengal| India|      2023-08-22|      true|
|      34| Customer_34|Hyderabad| Telangana| India|      2023-10-20|      true|
|      37| Customer_37|Hyderabad|  Gujarat| India|      2023-03-13|     false|
|      38| Customer_38|Hyderabad| Karnataka| India|      2023-06-19|     false|
|      40| Customer_40|Hyderabad|Maharashtra| India|      2023-07-29|     false|
|      44| Customer_44|Hyderabad| Telangana| India|      2023-08-18|     false|
|      84| Customer_84|Hyderabad|Maharashtra| India|      2023-04-08|     false|
|     100|Customer_100|Hyderabad|Maharashtra| India|      2023-12-30|     false|
|     110|Customer_110|Hyderabad|Maharashtra| India|      2023-03-14|     false|
|     118|Customer_118|Hyderabad|  Gujarat| India|      2023-01-27|     false|
|     134|Customer_134|Hyderabad|West Bengal| India|      2023-06-25|      true|
|     137|Customer_137|Hyderabad| Tamil Nadu| India|      2023-03-11|      true|
|     138|Customer_138|Hyderabad|    Delhi| India|      2023-12-26|      true|
|     149|Customer_149|Hyderabad| Karnataka| India|      2023-09-21|     false|
|     150|Customer_150|Hyderabad|Maharashtra| India|      2023-11-10|     false|
|     171|Customer_171|Hyderabad|West Bengal| India|      2023-12-24|      true|
|     173|Customer_173|Hyderabad|  Gujarat| India|      2023-05-30|     false|
+-----+-----+-----+-----+-----+-----+
only showing top 20 rows

```

```
customer_df.where(customer_df.city=="Hyderabad").show()
```

```

|      25| Customer_25|Hyderabad|West Bengal| India|      2023-08-22|      true|
|      34| Customer_34|Hyderabad| Telangana| India|      2023-10-20|      true|
|      37| Customer_37|Hyderabad|  Gujarat| India|      2023-03-13|     false|
|      38| Customer_38|Hyderabad| Karnataka| India|      2023-06-19|     false|

```

```

|      134|Customer_134|Hyderabad|West Bengal| India|      2023-06-25|      true|
|      137|Customer_137|Hyderabad| Tamil Nadu| India|      2023-03-11|      true|
|      138|Customer_138|Hyderabad|      Delhi| India|      2023-12-26|      true|
|      149|Customer_149|Hyderabad| Karnataka| India|      2023-09-21|     false|
|      150|Customer_150|Hyderabad|Maharashtra| India|      2023-11-10|     false|
|      171|Customer_171|Hyderabad|West Bengal| India|      2023-12-24|      true|
|      173|Customer_173|Hyderabad|  Gujarat| India|      2023-05-30|     false|
+-----+-----+-----+-----+-----+-----+-----+

```

only showing top 20 rows

5. Create or replace new column

The **withColumn** method is used to create a new column or replace an existing column in a DataFrame.

```
df.withColumn("name", "defination")
```

```

%python
from pyspark.sql.functions import col, concat, lit

# col: A function to reference a column in a DataFrame.
# concat: A function to concatenate multiple columns or strings.
# lit: A function to create a column with a literal value.

# Example: Adding a new column
df_with_new_column = customer_df.withColumn("full name", concat(col("name"), lit(" Singh")))

# Display the DataFrame
df_with_new_column.show()

```

```

▶ df_with_new_column: pyspark.sql.dataframe.DataFrame = [customer_id: integer, name: string ... 6 more fields]
|      2| Customer_2| Delhi|Maharashtra| India|      2023-08-05|      true| Customer_2 Singh|
|      3| Customer_3| Mumbai| Telangana| India|      2023-06-04|      true| Customer_3 Singh|
|      4| Customer_4| Delhi| Karnataka| India|      2023-03-15|     false| Customer_4 Singh|
|      5| Customer_5| Kolkata|West Bengal| India|      2023-08-19|      true| Customer_5 Singh|
|      6| Customer_6| Kolkata| Tamil Nadu| India|      2023-04-21|     false| Customer_6 Singh|
|      7| Customer_7| Mumbai| Telangana| India|      2023-05-23|      true| Customer_7 Singh|
|      8| Customer_8| Pune| Tamil Nadu| India|      2023-07-17|      true| Customer_8 Singh|
|      9| Customer_9| Delhi| Karnataka| India|      2023-06-02|      true| Customer_9 Singh|
|     10|Customer_10|Hyderabad| Delhi| India|      2023-02-23|      true|Customer_10 Singh|
|     11|Customer_11| Delhi|West Bengal| India|      2023-11-08|      true|Customer_11 Singh|
|     12|Customer_12| Delhi| Delhi| India|      2023-06-27|     false|Customer_12 Singh|
|     13|Customer_13| Pune|Maharashtra| India|      2023-02-03|      true|Customer_13 Singh|
|     14|Customer_14| Chennai| Karnataka| India|      2023-04-06|      true|Customer_14 Singh|
|     15|Customer_15|Hyderabad|West Bengal| India|      2023-03-31|      true|Customer_15 Singh|
|     16|Customer_16| Chennai|Maharashtra| India|      2023-04-26|      true|Customer_16 Singh|
|     17|Customer_17| Pune| Delhi| India|      2023-04-14|     false|Customer_17 Singh|
|     18|Customer_18| Chennai|Maharashtra| India|      2023-02-04|     false|Customer_18 Singh|
|     19|Customer_19| Chennai| Karnataka| India|      2023-01-22|      true|Customer_19 Singh|
+-----+-----+-----+-----+-----+-----+-----+

```

only showing top 20 rows

withColumnRenamed

- The withColumnRenamed method is used to rename a single column in a DataFrame.


```
%python
# Example: Renaming a column
df_renamed_column = df_with_new_column.withColumnRenamed("full name", "Full Name")

# Display the DataFrame
df_renamed_column.show()
```

```
▶ df_renamed_column: pyspark.sql.dataframe.DataFrame = [customer_id: integer, name: string ... 6 more fields]
```

2	Customer_2	Delhi	Maharashtra	India	2023-08-05	true	Customer_2 Singh
3	Customer_3	Mumbai	Telangana	India	2023-06-04	true	Customer_3 Singh
4	Customer_4	Delhi	Karnataka	India	2023-03-15	false	Customer_4 Singh
5	Customer_5	Kolkata	West Bengal	India	2023-08-19	true	Customer_5 Singh
6	Customer_6	Kolkata	Tamil Nadu	India	2023-04-21	false	Customer_6 Singh
7	Customer_7	Mumbai	Telangana	India	2023-05-23	true	Customer_7 Singh
8	Customer_8	Pune	Tamil Nadu	India	2023-07-17	true	Customer_8 Singh
9	Customer_9	Delhi	Karnataka	India	2023-06-02	true	Customer_9 Singh
10	Customer_10	Hyderabad	Delhi	India	2023-02-23	true	Customer_10 Singh
11	Customer_11	Delhi	West Bengal	India	2023-11-08	true	Customer_11 Singh
12	Customer_12	Delhi	Delhi	India	2023-06-27	false	Customer_12 Singh
13	Customer_13	Pune	Maharashtra	India	2023-02-03	true	Customer_13 Singh
14	Customer_14	Chennai	Karnataka	India	2023-04-06	true	Customer_14 Singh
15	Customer_15	Hyderabad	West Bengal	India	2023-03-31	true	Customer_15 Singh
16	Customer_16	Chennai	Maharashtra	India	2023-04-26	true	Customer_16 Singh
17	Customer_17	Pune	Delhi	India	2023-04-14	false	Customer_17 Singh
18	Customer_18	Chennai	Maharashtra	India	2023-02-04	false	Customer_18 Singh
19	Customer_19	Chennai	Karnataka	India	2023-01-22	true	Customer_19 Singh

only showing top 20 rows

6. Dropping a Column

The drop method is used to remove one or more columns from a DataFrame.

```
# Dropping a single column
df_dropped_column = df_renamed_column.drop("Full Name")

# Display the DataFrame
df_dropped_column.show()
```

```
▶ df_dropped_column: pyspark.sql.dataframe.DataFrame = [customer_id: integer, name: string ... 5 more fields]
```

2	Customer_2	Delhi	Maharashtra	India	2023-08-05	true
3	Customer_3	Mumbai	Telangana	India	2023-06-04	true
4	Customer_4	Delhi	Karnataka	India	2023-03-15	false
5	Customer_5	Kolkata	West Bengal	India	2023-08-19	true
6	Customer_6	Kolkata	Tamil Nadu	India	2023-04-21	false
7	Customer_7	Mumbai	Telangana	India	2023-05-23	true
8	Customer_8	Pune	Tamil Nadu	India	2023-07-17	true
9	Customer_9	Delhi	Karnataka	India	2023-06-02	true
10	Customer_10	Hyderabad	Delhi	India	2023-02-23	true
11	Customer_11	Delhi	West Bengal	India	2023-11-08	true

```

+-----+-----+-----+-----+-----+-----+
|      18|Customer_18| Chennai|Maharashtra| India|      2023-02-04| false|
|      19|Customer_19| Chennai| Karnataka| India|      2023-01-22|  true|
+-----+-----+-----+-----+-----+-----+
only showing top 20 rows

```

Dropping Multiple Columns

```

%python
# Dropping multiple columns
df_dropped_columns = df_renamed_column.drop("name", "country")

# Display the DataFrame
df_dropped_columns.show()

```

▶ df_dropped_columns: pyspark.sql.dataframe.DataFrame = [customer_id: integer, city: string ... 4 more fields]

```

+-----+-----+-----+-----+-----+-----+
|      2| Delhi|Maharashtra|      2023-08-05|  true| Customer_2 Singh|
|      3| Mumbai| Telangana|      2023-06-04|  true| Customer_3 Singh|
|      4| Delhi| Karnataka|      2023-03-15| false| Customer_4 Singh|
|      5| Kolkata|West Bengal|      2023-08-19|  true| Customer_5 Singh|
|      6| Kolkata| Tamil Nadu|      2023-04-21| false| Customer_6 Singh|
|      7| Mumbai| Telangana|      2023-05-23|  true| Customer_7 Singh|
|      8| Pune| Tamil Nadu|      2023-07-17|  true| Customer_8 Singh|
|      9| Delhi| Karnataka|      2023-06-02|  true| Customer_9 Singh|
|     10|Hyderabad| Delhi|      2023-02-23|  true|Customer_10 Singh|
|     11| Delhi|West Bengal|      2023-11-08|  true|Customer_11 Singh|
|     12| Delhi| Delhi|      2023-06-27| false|Customer_12 Singh|
|     13| Pune|Maharashtra|      2023-02-03|  true|Customer_13 Singh|
|     14| Chennai| Karnataka|      2023-04-06|  true|Customer_14 Singh|
|     15|Hyderabad|West Bengal|      2023-03-31|  true|Customer_15 Singh|
|     16| Chennai|Maharashtra|      2023-04-26|  true|Customer_16 Singh|
|     17| Pune| Delhi|      2023-04-14| false|Customer_17 Singh|
|     18| Chennai|Maharashtra|      2023-02-04| false|Customer_18 Singh|
|     19| Chennai| Karnataka|      2023-01-22|  true|Customer_19 Singh|
+-----+-----+-----+-----+-----+-----+

```

only showing top 20 rows

7. Removing Duplicate Rows

```

%python
# Removing duplicate rows
df_distinct = df_renamed_column.distinct()

# Display the DataFrame
df_distinct.show()

```

▶ df_distinct: pyspark.sql.dataframe.DataFrame = [customer_id: integer, name: string ... 6 more fields]

```

+-----+-----+-----+-----+-----+-----+
|      5| Customer_5| Kolkata|West Bengal| India|      2023-08-19|  true| Customer_5 Singh|
|      6| Customer_6| Kolkata| Tamil Nadu| India|      2023-04-21| false| Customer_6 Singh|
+-----+-----+-----+-----+-----+-----+

```

```

|      3| Customer_3| Mumbai| Telangana| India| 2023-06-04| true| Customer_3 Singh|
|     16| Customer_16| Chennai| Maharashtra| India| 2023-04-26| true| Customer_16 Singh|
|     12| Customer_12| Delhi| Delhi| India| 2023-06-27| false| Customer_12 Singh|
|     20| Customer_20| Pune| Karnataka| India| 2023-02-19| false| Customer_20 Singh|
|     11| Customer_11| Delhi| West Bengal| India| 2023-11-08| true| Customer_11 Singh|
|      4| Customer_4| Delhi| Karnataka| India| 2023-03-15| false| Customer_4 Singh|
|     19| Customer_19| Chennai| Karnataka| India| 2023-01-22| true| Customer_19 Singh|
|      7| Customer_7| Mumbai| Telangana| India| 2023-05-23| true| Customer_7 Singh|
|     14| Customer_14| Chennai| Karnataka| India| 2023-04-06| true| Customer_14 Singh|
|      1| Customer_1| Pune| West Bengal| India| 2023-08-10| true| Customer_1 Singh|
|     13| Customer_13| Pune| Maharashtra| India| 2023-02-03| true| Customer_13 Singh|
+-----+-----+-----+-----+-----+-----+-----+
only showing top 20 rows

```

Aggregation

- Will cover in detail tomorrow

```

+-----+-----+
|    city| count|
+-----+-----+
|Bangalore|661013|
|  Chennai|660249|
|   Mumbai|661241|
|Ahmedabad|660218|
|   Kolkata|660174|
|      Pune|660737|
|    Delhi|661025|
|Hyderabad|662281|
+-----+-----+

```